

MODULE 3

Bottom-Up Parsing: Shift Reduce parsing – Operator precedence parsing (Concepts only)

LR parsing – Constructing SLR parsing tables, Constructing, Canonical LR parsing tables and Constructing LALR parsing tables.

BOTTOM UP PARSING

A bottom-up parse starts with the string of terminals itself and builds from the leaves upward, working backwards to the start symbol by applying the productions in reverse.

Along the way, a bottom-up parser searches for substrings of the working string that match the right side of some production.

When it finds such a substring, it *reduces* it, i.e., substitutes the left side non-terminal for the matching right side. The goal is to reduce all the way up to the start symbol and report a successful parse.

SHIFT REDUCE PARSING

- ✚ Shift reduce parsing attempts to construct a parse tree for an input string beginning at the leaves (bottom) and working up towards the root (top).
- ✚ This can be considered as the process of “reducing” a string w to the start symbol of a grammar.
- ✚ At each reduction step parser searches for substrings of the working string that match the right side of some production.
- ✚ When it finds such a substring, it reduces it, i.e., substitutes the left side non-terminal for the matching right side. The goal is to reduce all the way up to the start symbol and report a successful parse.

- ✚ In Shift-reduce parsing a stack holds grammar symbols and an input buffer holds the rest of the string to be parsed.
- ✚ As we shall see, the handle always appears at the top of the stack just before it is identified as the handle.
- ✚ We use \$ to mark the bottom of the stack and also the right end of the input. Initially, the stack is empty, and the string w is on the input, as follows:

STACK	INPUT
\$	w \$

- ✚ During a left-to-right scan of the input string, the parser shifts zero or more input symbols onto the stack, until it is ready to reduce a string β of grammar symbols on top of the stack.
- ✚ It then reduces β to the head of the appropriate production. The parser repeats this cycle until it has detected an error or until the stack contains the start symbol and the input is empty as follows:

STACK	INPUT
\$ S	\$

- ✚ Upon entering this configuration, the parser halts and announces successful completion of parsing.

✚ There are actually four possible actions a shift-reduce parser can make:

- **Shift**
- **Reduce**
- **Accept**
- **Error**

1. **Shift:** The next input symbol is shifted onto the top of the stack.
2. **Reduce:** The parser knows the right end of the string to be reduced must be at the top of the stack. It must then locate the left end of the string within the stack and decide with what nonterminal to replace the string.
3. **Accept:** Announce successful completion of parsing.
4. **Error:** Discover a syntax error has occurred and calls an error recovery routine.

EXAMPLE

Following figure steps through the actions a shift-reduce parser might take in parsing the input string $\text{id}_1 * \text{id}_2$ according to the expression grammar.

$$E \rightarrow E + T \mid T$$

$$T \rightarrow T * F \mid F$$

$$F \rightarrow (E) \mid \text{id}$$

STACK	INPUT	ACTION
\$	id₁ * id₂ \$	shift
\$ id₁	* id₂ \$	reduce by $F \rightarrow \text{id}$
\$ F	* id₂ \$	reduce by $T \rightarrow F$
\$ T	* id₂ \$	shift
\$ T *	id₂ \$	shift
\$ T * id₂	\$	reduce by $F \rightarrow \text{id}$
\$ T * F	\$	reduce by $T \rightarrow T * F$
\$ T	\$	reduce by $E \rightarrow T$
\$ E	\$	accept

*Configurations of a shift-reduce parser on input $\text{id}_1 * \text{id}_2$*

OPERATOR PRECEDENCE PARSING

- Bottom-up parsers for a large class of context-free grammars can be easily developed using operator grammars.
 - In an operator grammar, no production rule can have:
 - No production right side is ϵ
 - No production right side has two adjacent nonterminal.
 - This property enables the implementation of efficient *operator-precedence parsers*.
- Variant of shift reduce parser.
 - Can be performed on ambiguous grammars.

EXAMPLE

$E \rightarrow AB$

$A \rightarrow a$

$B \rightarrow b$

not operator grammar

$E \rightarrow E + E \mid$

$E * E \mid$

$E / E \mid \text{id}$

operator grammar

Precedence Relations

- In operator-precedence parsing, we define three precedence relations between certain pairs of terminals as follows:

Relation	Meaning
$a <\cdot b$	a yields precedence to b (b has higher precedence than a)
$a =\cdot b$	a has the same precedence as b
$a \cdot > b$	a takes precedence over b (b has lower precedence than a)

- The intention of the precedence relations is to find the handle of a right sentential form,
 - $<\cdot$ with marking the left end,
 - $=\cdot$ appearing in the interior of the handle, and
 - $\cdot >$ marking the right end.
- In our input string $a_1a_2\dots a_n$, we insert the precedence relation between the pairs of terminals.

Steps:

1. Define precedence relations by constructing an **operator precedence table**.
2. **Insert dollar symbol** at beginning and end of the input string.
3. **Insert precedence operator** between every 2 symbols in the string by referring operator precedence table.
4. **Scan the string from left end until the first $.>$ is encountered.**
5. Then **scan backwards to the left until a $<.$ is encountered.**
6. **The handle contains everything to the left of the first $.>$ and to the right of the $<.$ encountered in step 5.**
7. **Reduce the handle using the LHS of appropriate production.**
8. **Repeat steps 4-7 until start symbol is reached.**

✚ Example: Consider the string $id + id * id$ and the grammar is:

$$E \rightarrow E + E \mid E - E \mid E * E \mid E / E \mid (E) \mid -E \mid id$$

✚ The corresponding precedence relations is

Step 1:

	id	+	*	\$
Id		.>	.>	.>
+	<.	.>	<.	.>
*	<.	.>	.>	.>
\$	<.	<.	<.	ACCEPT

✚ Then the string with the precedence relations inserted is:

Step 3: \$ <. id .> + <. id .> * <. id .> \$

✚ <. is inserted between the leftmost \$ and id since <. is the entry in row \$ and column id.

Precedence and associativity:
high to low

1. Id
2. ^ (R to L – represented as <.)
3. *, / (L to R – represented as .>)
4. +, - (L to R – represented as .>)
5. \$

Step 2:

\$ id + id * id \$

$\$ <. \text{id} .> + <. \text{id} .> * <. \text{id} .> \$$

Step 4: Scan from left to right

$\$ <. \text{id} .> + <. \text{id} .> * <. \text{id} .> \$$

$\$E+ <. \text{id} .> * <. \text{id} .> \$$

$\$E+ <. \text{id} .> * <. \text{id} .> \$$

$\$E+ E* <. \text{id} .> \$$

$\$E+ E* E \$$

The parsing decisions is not affected by non terminals hence can remove them.

$\$ + * \$$

Insert precedence operator in between symbols referring precedence operator table:

$\$ <. + <. * .> \$$

Scan from left to right:

$\$ <. + <. * .> \$$

$\$ <. + .> \$$

$\$ \$$

Handle And Handle Pruning

- ✚ In our input string $a_1 a_2 \dots a_n$, we insert the precedence relation between the pairs of terminals.
- ✚ Example: Consider the string $id + id * id$ and the grammar is:
- ✚ Handle is a substring that matches with the right side of the production and if the substring matches with the right side of the production, it is reduced with the non-terminal on the left side of the production

EXAMPLE

Consider the grammar

$E \rightarrow E + E$

$E \rightarrow E * E$

$E \rightarrow (E)$

$E \rightarrow id$

And the input string is $id_1 + id_2 * id_3$

The rightmost derivation is:

$E \rightarrow \underline{E + E}$

$\rightarrow E + \underline{E * E}$

$\rightarrow E + E * \underline{id_3}$

$\rightarrow E + \underline{id_2} * id_3$

$\rightarrow \underline{id_1} + id_2 * id_3$

In the above derivation, the underlined substrings are called handles.

HANDLE PRUNING

The process of obtaining rightmost derivation in reverse order is called “handle pruning”.
(i.e.) if w is a sentence or string of the grammar at hand, then $w = \gamma_n$ where γ_n is the n^{th} right sentinel form of some rightmost derivation.

ADVANTAGES

- Simple and easy to implement.

DISADVANTAGES

- It is hard to handle tokens like minus sign, which has different precedence (depending on whether it is unary or binary).
- Only small class of grammars (operator grammar) can be parsed using operator precedence techniques.

LR PARSING

LR parsing is most efficient method of bottom up parsing which can be used to parse large class of context free grammar.

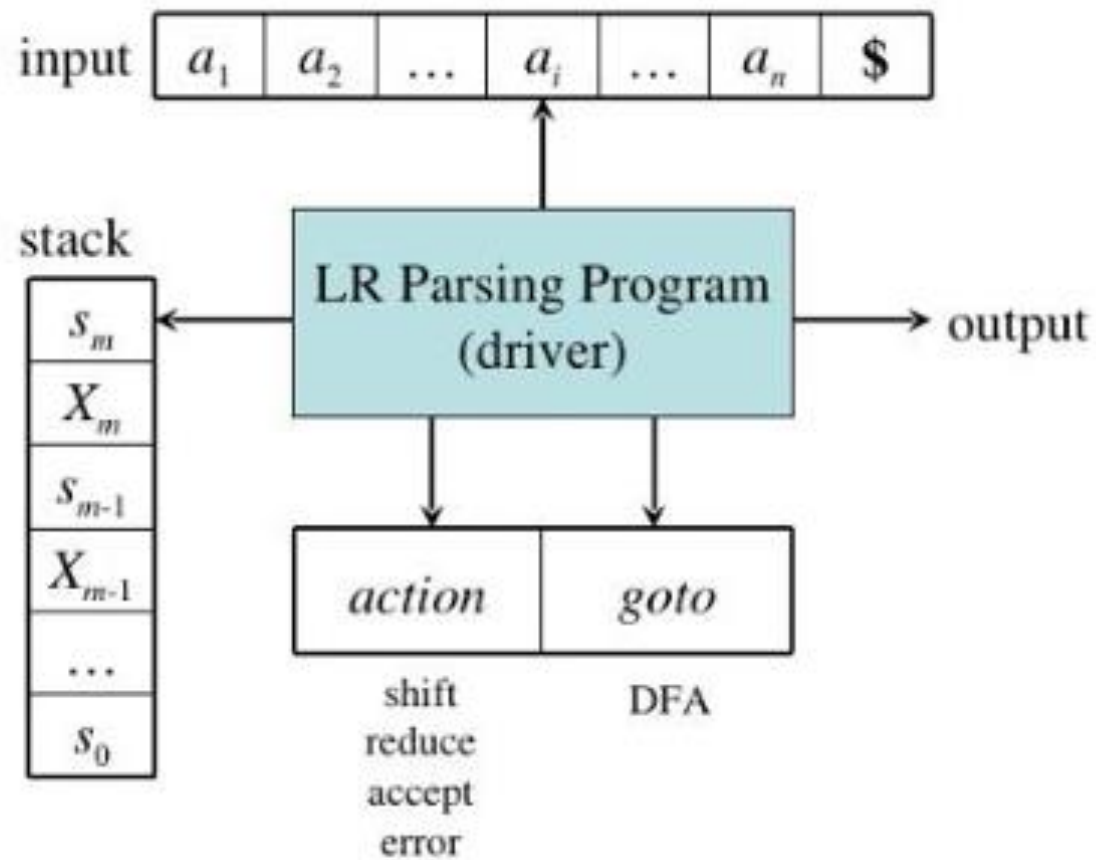
The technique is called LR(k) parsing; the “L” is for left to right scanning of input symbol, the “R” for constructing right most derivation in reverse, and the k for the number of input symbols of lookahead that are used in making parsing decision.

There are three types of LR parsing,

- SLR (Simple LR)
- CLR (Canonical LR)
- LALR (Lookahead LR)

The schematic form of LR parser is given below.

Model of an LR Parser



Requirements of LR parser:

- Input Buffer
- Stack
- Parsing Table
- LR Parsing Program

Input Buffer

The parsing program reads characters from an input buffer one at a time.

Stack

- ✚ Parsing program uses a stack to store string of the form $s_0X_1s_1X_2s_2X_3 \dots\dots\dots X_ms_m$ where s_m is on the top.
- ✚ X_i is a grammar symbol and S_i is the state
- ✚ Each state symbol summarizes the information contained in the stack below it
- ✚ Combination of state symbol on stack top and current input symbol are used to index the parsing table and determine the shift reduce parsing decision

Parsing Table

Consist of 2 parts

1. parsing action function action
2. goto function goto

Action

This function takes as arguments a state i and a terminal a (or \$, the input end marker). The value of ACTION $[i, a]$ can have one of the four forms:

- i. Shift j , where j is a state.
- ii. Reduce by a grammar production $A \rightarrow \beta$.
- iii. Accept.
- iv. Error.

Goto

This function takes a state and grammar symbol as arguments and produces a state. If $GOTO [i, A] = j$, the GOTO also maps a state i and nonterminal A to state j .

Parsing Program works as follows

- ✚ It determines s_m , the state currently on top of stack and a_i the current input symbol
- ✚ It consults parsing action table entry for $\text{action}[s_m, a_i]$ which can have 4 values:
- ✚ The function goto takes a state and grammar symbol as arguments and produces a state.
 1. **Shift s , where s is a state**
 2. **Reduce by a grammar production $A \rightarrow b$**
 3. **Accept**
 4. **Error**

- ✚ The function goto takes a state and grammar symbol as arguments and produces a state.
- ✚ A configuration of LR parser is a pair whose first component is stack contents and second component is remaining input:

$$(s_0 X_1 s_1 X_2 \dots X_m s_m, a_i a_{i+1} \dots a_n \$)$$

- ✚ The next move of the parser is determined by reading a_i , the current input symbol and s_m , state on stack top and then consulting parsing action table entry $\text{action}[s_m, a_i]$.

✚ The configuration resulting after each of four types of moves are as follows:

1. If action $[s_m, a_i] = \text{shift } s$, the parser executes a shift move, entering the configuration

$(S_0 X_1 S_1 X_2 \dots\dots\dots X_m S_m a_i S, a_{i+1} \dots\dots\dots a_n \$)$

Here the parser has shifted both the current input symbol a_i and the next state s , which is given in action $[s_m, a_i]$, onto the slack; a_{i+1} becomes the current input symbol.

2. If action $[s_m, a_i] = \text{reduce } A \rightarrow \beta$, then the parser executes a reduce move, entering the configuration

$$(s_0 X_1 s_1 X_2 s_2 \dots X_{m-r} s_{m-r} A s, a_i a_{i+1} \dots a_n \$)$$

where $s = \text{goto}[s_{m-r}, A]$ and r is the length of β , the right side of the production. Here the parser first popped $2r$ symbols off the stack (r state symbols and r grammar symbols), exposing state s_{m-r} . The parser then pushed both A , the left side of the production, and s , the entry for $\text{goto}[s_{m-r}, A]$, onto the stack. The current input symbol is not changed in a reduce move.

3. If action $[s_m, a_i] = \text{accept}$, parsing is completed.
4. If action $[s_m, a_i] = \text{error}$, the parser has discovered an error and calls an error recovery routine.

LR Parsing Algorithm

INPUT : Input string w , LR-Parsing table with functions ACTION and GOTO for a grammar G

OUTPUT : If w is in $L(G)$, a bottom-up parse for w , otherwise, an error indication.

METHOD : Initially, the parser has s_0 on its stack, where s_0 is the initial state, and $w\$$ in the input buffer. The parser then executes the following program until an accept or error action is encountered.


```

set ip to point to the first symbol of  $w\$$ 
repeat forever begin
    Let s be the state on top of the stack and
        a the symbol pointed to by ip;
    if action [s, a] = shift s' then begin
        push a then s' on top of the stack;
        advance ip to the next input symbol
    end
    else if action [s, a] = reduce  $A \rightarrow \beta$  then begin
        pop  $2 * |\beta|$  symbols off the stack ;
        let s' be the state now on top of the stack;
        push A then goto [s', A] on top of the stack;
        output the production  $A \rightarrow \beta$ 
    end
    else if action [s, a] = accept then
        return
    else error();
end

```

Example:

$$(1) \quad E \rightarrow E + T$$

$$(2) \quad E \rightarrow T$$

$$(3) \quad T \rightarrow T * F$$

$$(4) \quad T \rightarrow F$$

$$(5) \quad F \rightarrow (E)$$

$$(6) \quad F \rightarrow \text{id}$$

STATE	ACTION						GOTO		
	id	+	*	(	)	\$	<i>E</i>	<i>T</i>	<i>F</i>
0	s5			s4			1	2	3
1		s6				acc			
2		r2	s7		r2	r2			
3		r4	r4		r4	r4			
4	s5			s4			8	2	3
5		r6	r6		r6	r6			
6	s5			s4				9	3
7	s5			s4					10
8		s6			s11				
9		r1	s7		r1	r1			
10		r3	r3		r3	r3			
11		r5	r5		r5	r5			

STACK		INPUT	ACTION
(1)	0	id * id + id \$	shift
(2)	0 id 5	* id + id \$	reduce by $F \rightarrow id$
(3)	0 F 3	* id + id \$	reduce by $T \rightarrow F$
(4)	0 T 2	* id + id \$	shift
(5)	0 T 2 * 7	id + id \$	shift
(6)	0 T 2 * 7 id 5	+ id \$	reduce by $F \rightarrow id$
(7)	0 T 2 * 7 F 10	+ id \$	reduce by $T \rightarrow T * F$
(8)	0 T 2	+ id \$	reduce by $E \rightarrow T$
(9)	0 E 1	+ id \$	shift
(10)	0 E 1 + 6	id \$	shift
(11)	0 E 1 + 6 id 5	\$	reduce by $F \rightarrow id$
(12)	0 E 1 + 6 F 3	\$	reduce by $T \rightarrow F$
(13)	0 E 1 + 6 T 9	\$	$E \rightarrow E + T$
(14)	0 E 1	\$	accept

Moves of LR parser on **id * id + id.**

Different types of LR Parsers

All LR parsers use the same parsing algorithm but the difference is only in terms of the construction of parsing tables. There are three widely used algorithms available for constructing an LR parsing Tables. Different types of LR parsers are:

✚ SLR(l) - Simple LR

- Works on smallest class of grammar.
- Few number of states, hence very small table.
- Simple and fast construction.

✚ LR(1) - LR parser

- Also called as Canonical LR parser.
- Works on complete set of LR(1) Grammar.
- Generates large table and large number of states.
- Slow construction.

LALR(1) - Look ahead LR parser

- Works on intermediate size of grammar.
- Number of states are same as in SLR(1).

Reasons for attractiveness of LR parser

-  LR parsers can handle a large class of context-free grammars.
-  The LR parsing method is a most general non-back tracking shift-reduce parsing method.
-  An LR parser can detect the syntax errors as soon as they can occur.
-  LR grammars can describe more languages than LL grammars.

Drawbacks of LR parsers

- ✚ It is too much work to construct LR parser by hand. It needs an automated parser generator.
- ✚ If the grammar contains ambiguities or other constructs then it is difficult to parse in a left-to-right scan of the input.

Construction of LR Parsing Table

3 techniques for constructing LR parsing table for a grammar :-

1. SLR
2. CLR
3. LALR

CONSTRUCTING SLR PARSING TABLES

LR(0) Item

- ✚ LR parser using SLR parsing table is called an SLR parser.
- ✚ A grammar for which an SLR parser can be constructed is an SLR grammar.
- ✚ An LR(0) item (item) of a grammar G is a production of G with a dot at the some position of the right side.

Ex: for the productions $A \rightarrow aBb$

Possible LR(0) Items are:

$A \rightarrow .aBb$

$A \rightarrow a.Bb$

$A \rightarrow aB.b$

$A \rightarrow aBb.$

- ✚ A production rule of the form $A \rightarrow \varepsilon$ yields only one item $A \rightarrow \cdot$.
- ✚ Intuitively, an item shows how much of a production we have seen till the current point in the parsing procedure.
- ✚ A collection of sets of LR(0) items (the canonical LR(0) collection) is the basis for constructing SLR parsers.
- ✚ To construct the canonical LR(0) collection for a grammar we define:
 - **Augmented grammar**
 - **Two functions: CLOSURE and GOTO**

Augmented grammar

- ✚ G' is the augmented grammar of G with a new production rule $S' \rightarrow S$ where S' is the new starting symbol i.e,

$$G' = G \cup \{S' \rightarrow S\} \text{ where } S \text{ is the start state of } G.$$

- ✚ The start state of $G' = S'$.
- ✚ This is done to signal to the parser when the parsing should stop to announce acceptance of input.

Kernel and Non-Kernel items

- ✚ Kernel items include the set of items that do not have the dot at leftmost end.
- ✚ $S' \rightarrow .S$ is an exception and is considered to be a kernel item.
- ✚ Non-kernel items are the items which have the dot at leftmost end.
- ✚ Sets of items are formed by taking the closure of a set of kernel items.

The Closure Operation

- ✚ If I is a set of LR(0) items for a grammar G , then $\text{closure}(I)$ is the set of LR(0) items constructed from I by the two rules:
 1. Initially, every LR(0) item in I is added to $\text{closure}(I)$.
 2. If $A \rightarrow \alpha . B \beta$ is in $\text{closure}(I)$ and $B \rightarrow \gamma$ is a production rule of G ; then $B \rightarrow . \gamma$ will be in the $\text{closure}(I)$.
- ✚ We will apply this rule until no more new LR(0) items can be added to $\text{closure}(I)$.

Computation of Closure

function closure (I)

begin

J := I;

repeat

for each item $A \rightarrow \alpha . B \beta$ in J and each production

$B \rightarrow \gamma$ of G such that $B \rightarrow . \gamma$ is not in J do

add $B \rightarrow . \gamma$ to J

until no more items can be added to J

return J

end

The Closure Operation -- Example

$E' \rightarrow E$

$E \rightarrow E+T$

$E \rightarrow T$

$T \rightarrow T*F$

$T \rightarrow F$

$F \rightarrow (E)$

$F \rightarrow id$

$\text{closure}(\{E' \rightarrow \bullet E\}) =$

$\{ E' \rightarrow \bullet E \} \leftarrow \text{kernel items}$

$E \rightarrow \bullet E+T$

$E \rightarrow \bullet T$

$T \rightarrow \bullet T*F$

$T \rightarrow \bullet F$

$F \rightarrow \bullet (E)$

$F \rightarrow \bullet id \}$

Goto Operation

- ✚ If I is a set of LR(0) items and X is a grammar symbol (terminal or non-terminal), then $\text{goto}(I, X)$ is defined as follows:
 - If $A \rightarrow \alpha . X\beta$ in I then every item in $\text{closure}(\{A \rightarrow \alpha . X\beta\})$ will be in $\text{goto}(I, X)$.
 - If I is the set of items that are valid for some viable prefix γ , then $\text{goto}(I, X)$ is the set of items that are valid for the viable prefix γX .

EXAMPLE

$$I = \{ E' \rightarrow E., E \rightarrow E.+T \}$$

$$\text{goto}(I, +) = \{ E \rightarrow E+.T \}$$

$$T \rightarrow .T * F$$

$$T \rightarrow .F$$

$$F \rightarrow .(E)$$

$$F \rightarrow .id \}$$

Constructing SLR Parsing Table

INPUT: An augmented grammar G' .

OUTPUT: The SLR parsing table functions *action* and *goto* for G' .

METHOD:

1. Construct $C = \{I_0, I_1, \dots, I_n\}$, the collection of sets of LR(0) items for G' .
2. State i is constructed from I_i . The parsing actions for state i are determined as follows:
 - (a) If $[A \rightarrow \alpha \cdot a \beta]$ is in I_i and $\text{goto}(I_i, a) = I_j$, then set $\text{action}[i, a]$ to “shift j ”. Here a must be terminal.
 - (b) If $[A \rightarrow \alpha \cdot]$ is in I_i , then set $\text{action}[i, a]$ to “reduce $A \rightarrow \alpha$ ” for all a in $\text{FOLLOW}(A)$.
 - (c) If $[S' \rightarrow S \cdot]$ is in I_i , then set $\text{action}[i, \$]$ to “accept”.

If any conflicting actions are generated by the above rules, we say the grammar is not SLR (1). The algorithm fails to produce a parser in this case.

3. The goto transitions for state i are constructed for all nonterminals A using the rule:
if $\text{GOTO}(I_i, A) = I_j$ then $\text{GOTO}[I, A] = j$.
4. All entries not defined by rules (2) and (3) are made "error".
5. The initial state of the parser is the one constructed from the set of items containing $[S' \rightarrow S]$.

Construction of Canonical LR(0) Collection

- ✚ To create the SLR parsing tables for a grammar G , we will create the canonical LR(0) collection of the grammar G' .

ALGORITHM

Procedure items(G')

begin

$C := \{ \text{closure}(\{S' \rightarrow .S\}) \}$

repeat for each set of items I in C and each grammar symbol X

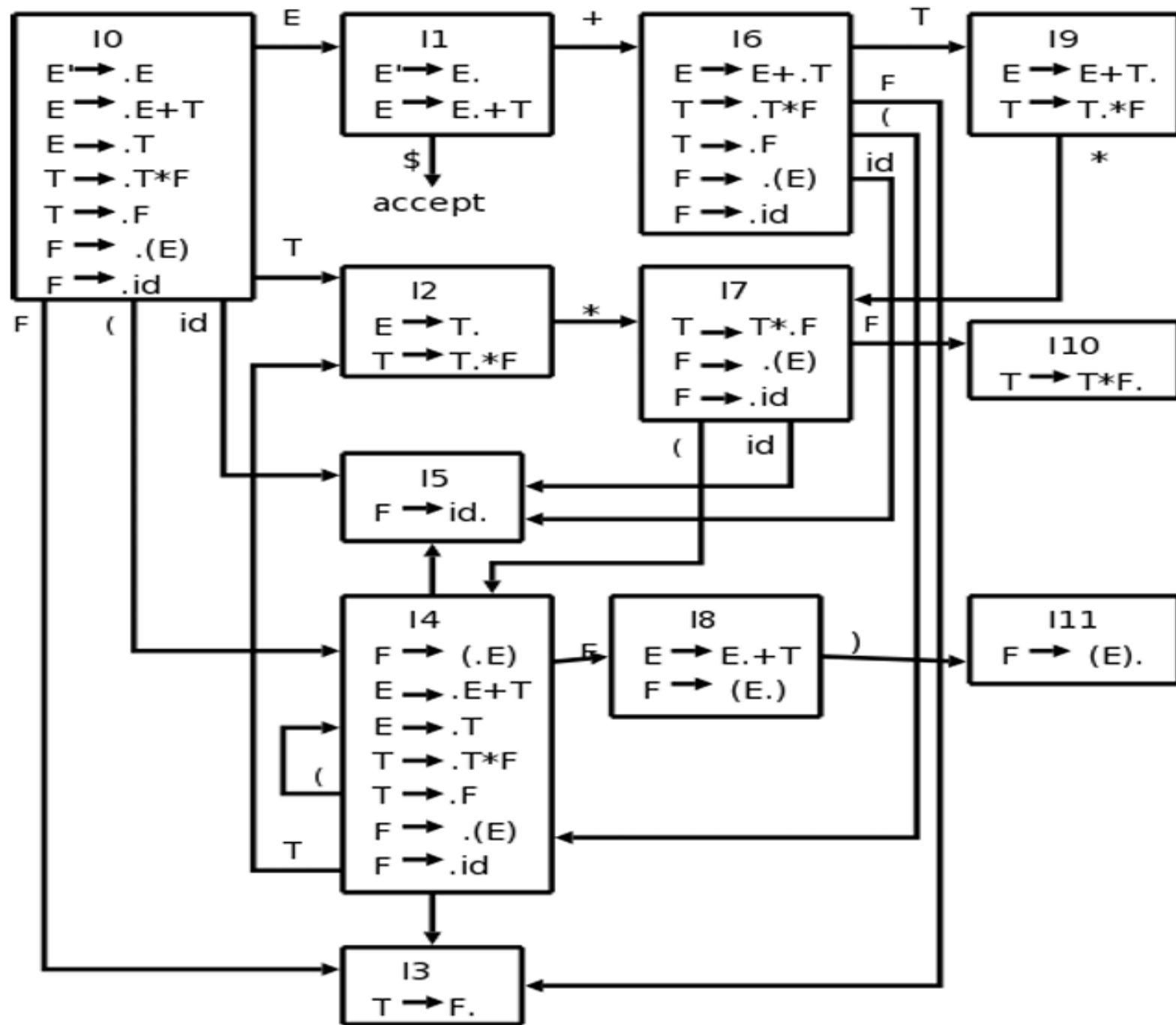
if goto(I, X) is not empty and not in C

add goto(I, X) to C

until no more set of LR(0) items can be added to C .

\end

- ✚ goto function is a DFA on the sets in C .



Parsing Tables of Expression Grammar

Action Table

Goto Table

state	id	+	*	(	)	\$		E	T	F
0	s5			s4				1	2	3
1		s6				acc				
2		r2	s7		r2	r2				
3		r4	r4		r4	r4				
4	s5			s4				8	2	3
5		r6	r6		r6	r6				
6	s5			s4					9	3
7	s5			s4						10
8		s6			s11					
9		r1	s7		r1	r1				
10		r3	r3		r3	r3				
11		r5	r5		r5	r5				

Blank entries are error entries.

Number the given Grammar

$E' \rightarrow E$

$E \rightarrow E+T$ -----(1)

$E \rightarrow T$ -----(2)

$T \rightarrow T * F$ -----(3)

$T \rightarrow F$ -----(4)

$F \rightarrow (E)$ -----(5)

$F \rightarrow id$ -----(6)

$FOLLOW(E) = \{ \$,), + \}$

$FOLLOW(T) = \{ \$, +,), * \}$

$FOLLOW(F) = \{ *, +,), \$ \}$

I₁: $E' \rightarrow E.$	(1, \$)	====> ACCEPT
I₂: $E \rightarrow T.$	(2, \$), (2,)), (2, +)	====> r2
I₃: $T \rightarrow F.$	(3, \$), (3, +), (3,)), (3, *)	====> r4
I₅: $F \rightarrow id.$	(5, *), (5, +), (5,)), (5, \$)	====> r6
I₉: $E \rightarrow E+T.$	(9, \$), (9,)), (9, +)	====> r1
I₁₀: $T \rightarrow T * F.$	(10, \$), (10, +), (10,)), (10, *)	====> r3
I₁₁: $F \rightarrow (E).$	(11, *), (11, +), (11,)), (11, \$)	====> r5

Shift/Reduce and Reduce/Reduce conflicts

- ✚ If a state does not know whether it will make a shift operation or reduction for a terminal, we say that there is a shift/reduce conflict.
- ✚ If a state does not know whether it will make a reduction operation using the production rule i or j for a terminal, we say that there is a reduce/reduce conflict.
- ✚ If the SLR parsing table of a grammar G has a conflict, we say that that grammar is not SLR grammar.

Stack Implementation

✚ Check whether the given input is valid or not?

Stack	Input	Action
\$ 0	id*id+id\$	shift 5
\$ 0 id 5	*id+id\$	reduce 6 goto 3
\$ 0 <i>F</i> 3	*id+id\$	reduce 4 goto 2
\$ 0 <i>T</i> 2	*id+id\$	shift 7
\$ 0 <i>T</i> 2 * 7	id+id\$	shift 5
\$ 0 <i>T</i> 2 * 7 id 5	+id\$	reduce 6 goto 10
\$ 0 <i>T</i> 2 * 7 <i>F</i> 10	+id\$	reduce 3 goto 2
\$ 0 <i>T</i> 2	+id\$	reduce 2 goto 1
\$ 0 <i>E</i> 1	+id\$	shift 6
\$ 0 <i>E</i> 1 + 6	id\$	shift 5
\$ 0 <i>E</i> 1 + 6 id 5	\$	reduce 6 goto 3
\$ 0 <i>E</i> 1 + 6 <i>F</i> 3	\$	reduce 4 goto 9
\$ 0 <i>E</i> 1 + 6 <i>T</i> 9	\$	reduce 1 goto 1
\$ 0 <i>E</i> 1	\$	accept

EXAMPLE

SLR (1) GRAMMAR

$S \rightarrow E$

$E \rightarrow E + T \mid T$

$T \rightarrow T * F \mid F$

$F \rightarrow \text{id}$

AUGMENTED GRAMMAR

$S' \rightarrow S$

$S \rightarrow E$

$E \rightarrow E + T \mid T$

$T \rightarrow T * F \mid F$

$F \rightarrow \text{id}$

CANONICAL COLLECTION OF LR(0) ITEMS

I_0

$S' \rightarrow S$

$S \rightarrow \bullet E$

$E \rightarrow \bullet E + T$

$E \rightarrow \bullet T$

$T \rightarrow \bullet T * F$

$T \rightarrow \bullet F$

$F \rightarrow \bullet \text{id}$

$I_1 = \text{Go to } (I_0, E)$

$S' \rightarrow E \bullet$

$E \rightarrow E \bullet + T$

$I_2 = \text{Go to } (I_0, T)$

$E \rightarrow T \bullet T$

$T \bullet \rightarrow * F$

$I_3 = \text{Go to } (I_0, F)$

$T \rightarrow F \bullet$

$I_4 = \text{Go to } (I_0, \text{id})$

$F \rightarrow \text{id} \bullet$

$I_5 = \text{Go to } (I_1, +)$

$E \rightarrow E + \bullet T$

$T \rightarrow \bullet T * F$

$T \rightarrow \bullet F$

$F \rightarrow \bullet \text{id}$

$I_6 = \text{Go to } (I_2, *)$

$T \rightarrow T * \bullet F$

$F \rightarrow \bullet \text{id}$

$I_7 = \text{Go to } (I_5, T)$

$E \rightarrow E + T \bullet$

$I_3 = \text{Go to } (I_5, F)$

$T \rightarrow F \bullet$

$I_4 = \text{Go to } (I_5, \text{id})$

$F \rightarrow \text{id} \bullet$

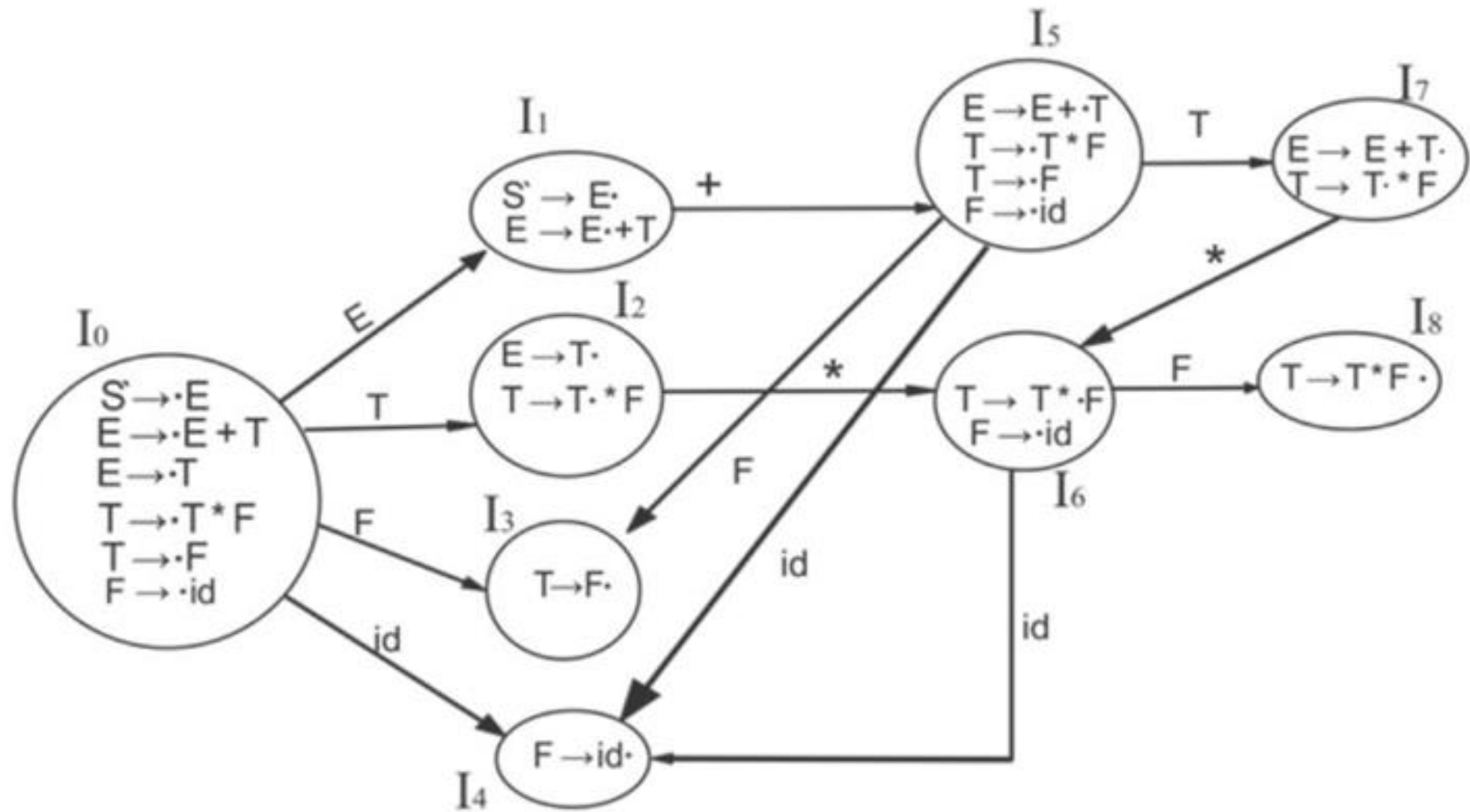
$I_8 = \text{Go to } (I_6, F)$

$T \rightarrow T * F \bullet$

$I_4 = \text{Go to } (I_6, \text{id})$

$F \rightarrow \text{id} \bullet$

DRAW CANONICAL COLLECTION OF LR(0) ITEMS



NUMBER THE PRODUCTIONS

1. $S \rightarrow E$
2. $E \rightarrow E + T \mid T$
3. $T \rightarrow T * F \mid F$
4. $F \rightarrow id$

SLR(1) TABLE

FOLLOW (S) = {\$}

FOLLOW (E) = {+, \$}

FOLLOW (T) = {*, +, \$}

FOLLOW (F) = {*, +, \$}

States	Action				Go to		
	id	+	*	\$	E	T	F
I ₀	S ₄				1	2	3
I ₁		S ₅		Accept			
I ₂		R ₂	S ₆	R ₂			
I ₃		R ₄	R ₄	R ₄			
I ₄		R ₅	R ₅	R ₅			
I ₅	S ₄					7	3
I ₆	S ₄						8
I ₇		R ₁	S ₆	R ₁			
I ₈		R ₃	R ₃	R ₃			

EXAMPLE

Every SLR (1) grammar is unambiguous, but there are many unambiguous grammars that are not SLR(1). Consider the grammar with productions

$$S \rightarrow L = R$$

$$S \rightarrow R$$

$$L \rightarrow * R$$

$$L \rightarrow \text{id}$$

$$R \rightarrow L$$

AUGMENTED GRAMMAR

$$S' \rightarrow S$$

$$S \rightarrow L = R$$

$$S \rightarrow R$$

$$L \rightarrow * R$$

$$L \rightarrow \text{id}$$

$$R \rightarrow L$$

CANONICAL COLLECTION OF LR(0) ITEMS

I₀

$S' \rightarrow \bullet S$

$S \rightarrow \bullet L = R$

$S \rightarrow \bullet R$

$L \rightarrow \bullet * R$

$L \rightarrow \bullet \text{id}$

$R \rightarrow \bullet L$

I₅ = Go to (I₀, id)

$L \rightarrow \text{id} \bullet$

I₆ = Go to (I₂, =)

$S \rightarrow L = \bullet R$

$R \rightarrow \bullet L$

$L \rightarrow \bullet * R$

$L \rightarrow \bullet \text{id}$

I₉ = Go to (I₆, R)

$S \rightarrow L = R \bullet$

I₈ = Go to (I₆, L)

$R \rightarrow L \bullet$

I₇ = Go to (I₆, L)

$L \rightarrow * R \bullet$

I₃ = Go to (I₀, R)

$S \rightarrow R \bullet$

I₄ = Go to (I₀, *)

$L \rightarrow * \bullet R$

$R \rightarrow \bullet L$

$L \rightarrow \bullet * R$

$L \rightarrow \bullet \text{id}$

I₄ = Go to (I₄, *)

$L \rightarrow * \bullet R$

$R \rightarrow \bullet L$

$L \rightarrow \bullet * R$

$L \rightarrow \bullet \text{id}$

I₅ = Go to (I₆, id)

$L \rightarrow \text{id} \bullet$

I₁ = Go to (I₀, S)

$S' \rightarrow S \bullet$

I₇ = Go to (I₄, R)

$L \rightarrow * R \bullet$

I₄ = Go to (I₆, *)

$L \rightarrow * \bullet R$

$R \rightarrow \bullet L$

$L \rightarrow \bullet * R$

$L \rightarrow \bullet \text{id}$

I₂ = Go to (I₀, L)

$S \rightarrow L \bullet = R$

$R \rightarrow L \bullet$

I₈ = Go to (I₄, L)

$R \rightarrow L \bullet$

I₅ = Go to (I₄, id)

$L \rightarrow \text{id} \bullet$

NUMBER THE PRODUCTIONS

1. $S \rightarrow L = R$
2. $S \rightarrow R$
3. $L \rightarrow * R$
4. $L \rightarrow \text{id}$
5. $R \rightarrow L$

PARSING TABLE

$I_1 = S' \rightarrow S \cdot$

$I_2 = R \rightarrow L \cdot$

$I_3 = S \rightarrow R \cdot$

$I_5 = L \rightarrow \text{id} \cdot$

$I_7 = L \rightarrow * R \cdot$

$I_8 = R \rightarrow L \cdot$

$I_9 = S \rightarrow L = R \cdot$

FOLLOW (S) = { \$ }

FOLLOW (L) = { =, \$ }

FOLLOW (R) = { =, \$ }

(2, =), (2, \$) =====> r5

(3, \$) =====> r2

(5, =), (5, \$) =====> r4

(7, =), (7, \$) =====> r3

(8, \$), (8, =) =====> r5

(9, \$) =====> r1

STATE	ACTION	GOTO
	id * = \$	R L S
0	s5 s4	3 2 1
1	ACCEPT	
2	s6/ r5 r5	
3	r2	
4	s5 s4	7 8
5	r4 r4	
6	s5 s4	9 8/7
7	r3 r3	
8	r5 r5	
9	r1	

Here there is a shift reduce conflict in (2,=) so grammar is not SLR(1)

CONSTRUCTING CANONICAL LR(CLR) PARSING TABLES

- Most general technique for constructing an LR parsing table from a grammar.
- It is possible to carry more information in the state that will allow us to rule out some of invalid reductions $A \rightarrow \alpha$
- By splitting states when necessary, we can arrange to have each state of an LR parser indicate exactly which input symbols can follow a handle for which there is a possible reduction to A .

- To avoid some of invalid reductions, the states need to carry more information.
- Extra information is put into a state by including a terminal symbol as a second component in an item.
- A LR(1) item is:

$$A \rightarrow \alpha.\beta,a \quad \text{where } \mathbf{a} \text{ is the look-head of the LR(1) item}$$

($\mathbf{a}$ is a terminal or end-marker.)

- Such an object is called LR(1) item.
 - 1 refers to the length of the second component
 - The lookahead has no effect in an item of the form $[A \rightarrow \alpha.\beta,a]$, where β is not ϵ .
 - But an item of the form $[A \rightarrow \alpha.,a]$ calls for a reduction by $A \rightarrow \alpha$ only if the next input symbol is a .

- When β (in the LR(1) item $A \rightarrow \alpha.\beta,a$) is not empty, the look-head does not have any affect.
- When β is empty ($A \rightarrow \alpha.,a$), we do the reduction by $A \rightarrow \alpha$ only if the next input symbol is a
- Set of a 's will be a subset of FOLLOW(A), but it could be a proper subset.

Canonical Collection of Sets of LR(1) Items

- ✚ The construction of the canonical collection of the sets of LR(1) items are similar to the construction of the canonical collection of the sets of LR(0) items, except that closure and goto operations work a little bit different.

goto operation

- If I is a set of LR(1) items and X is a grammar symbol (terminal or non-terminal), then $\text{goto}(I, X)$ is defined as follows:
 - If $A \rightarrow \alpha.X\beta, a$ in I then every item in $\text{closure}(\{A \rightarrow \alpha.X\beta, a\})$ will be in $\text{goto}(I, X)$.

closure(I) is: (where I is a set of LR(1) items)

- every LR(1) item in I is in $\text{closure}(I)$
- if $A \rightarrow \alpha.B\beta, a$ in $\text{closure}(I)$ and $B \rightarrow \gamma$ is a production rule of G ; then $B \rightarrow \gamma, b$ will be in the $\text{closure}(I)$ for each terminal b in $\text{FIRST}(\beta a)$.

Construction of CLR (canonical LR) Parsing Tables

Algorithm: Construction of canonical- LR parsing tables.

INPUT: An augmented grammar G' .

OUTPUT: the canonical-LR parsing table functions ACTION AND GOTO for G' .

METHOD:

1. Construct $C' = \{ I_0, I_1, \dots, I_n \}$, the collection of sets of LR(1) items for G' .
2. State i of the parser is constructed from I_i . The parsing action for state i is determined as follows.

(a) If $[A \rightarrow \alpha.a\beta, b]$ is in I_i , and $\text{GOTO}(I_i, a) = I_j$, then set ACTION $[i, a]$ To "shift j ". Here a must be a terminal.

(b) If $[A \rightarrow \alpha., a]$ is in I_i , $A \neq S'$, then set ACTION $[i, a]$ to "reduce $A \rightarrow \alpha.$ ".

(c) If $[S' \rightarrow S., \$]$ is in I_i , then set ACTION $[i, \$]$ to "accept".

If any conflicting actions result from the above rules, we say the grammar is not LR(1). The algorithm falls to produce a parser in this case.

3. The goto transitions for state i are constructed for all non terminals A using the rule: if $\text{GOTO}(I_i, A) = I_j$, then $\text{GOTO}[i, A] = j$.
4. All entries not defined by rules (2) and (3) are made "error".
5. The initial state of the parser is the one constructed from the set of items containing $[S' \rightarrow .S, \$]$.

EXAMPLE

Construct CLR parsing table for the following grammar

$S \rightarrow CC$

$C \rightarrow cC \mid d$

AUGMENTED GRAMMAR

$S' \rightarrow S$

$S \rightarrow CC$

$C \rightarrow cC$

$C \rightarrow d$

CANONICAL COLLECTION OF LR(1) ITEMS

I_0

$S' \rightarrow \bullet S, \$$

$S \rightarrow \bullet CC, \$$

$C \rightarrow \bullet cC, c \mid d$

$C \rightarrow \bullet d, c \mid d$

$I_1 = \text{GOTO}(I_0, S)$

$S' \rightarrow S\bullet, \$$

$I_2 = \text{GOTO}(I_0, C)$

$S \rightarrow C\bullet C, \$$

$C \rightarrow \bullet cC, \$$

$C \rightarrow \bullet d, \$$

$I_3 = \text{GOTO}(I_0, c)$

$C \rightarrow c\bullet C, c \mid d$

$C \rightarrow \bullet cC, c \mid d$

$C \rightarrow \bullet d, c \mid d$

$I_4 = \text{GOTO}(I_0, d)$

$C \rightarrow d\bullet, c \mid d$

$I_5 = \text{GOTO}(I_2, C)$

$S \rightarrow CC\bullet, \$$

$I_6 = \text{GOTO}(I_2, c)$

$C \rightarrow c\bullet C, \$$

$C \rightarrow \bullet cC, \$$

$C \rightarrow \bullet d, \$$

$I_7 = \text{GOTO}(I_2, d)$

$C \rightarrow d\bullet, \$$

$I_4 = \text{GOTO}(I_0, d)$

$C \rightarrow d\bullet, c \mid d$

$I_5 = \text{GOTO}(I_2, C)$

$S \rightarrow CC\bullet, \$$

$I_6 = \text{GOTO}(I_2, c)$

$C \rightarrow c\bullet C, \$$

$C \rightarrow \bullet cC, \$$

$C \rightarrow \bullet d, \$$

$I_7 = \text{GOTO}(I_2, d)$

$C \rightarrow d\bullet, \$$

$I_8 = \text{GOTO}(I_3, C)$

$C \rightarrow cC\bullet, c \mid d$

$I_3 = \text{GOTO}(I_3, c)$

$C \rightarrow c\bullet C, c \mid d$

$C \rightarrow \bullet cC, c \mid d$

$C \rightarrow \bullet d, c \mid d$

$I_4 = \text{GOTO}(I_3, d)$

$C \rightarrow d\bullet, c \mid d$

$I_9 = \text{GOTO}(I_6, C)$

$C \rightarrow cC\bullet, \$$

$I_6 = \text{GOTO}(I_6, c)$

$C \rightarrow c\bullet C, \$$

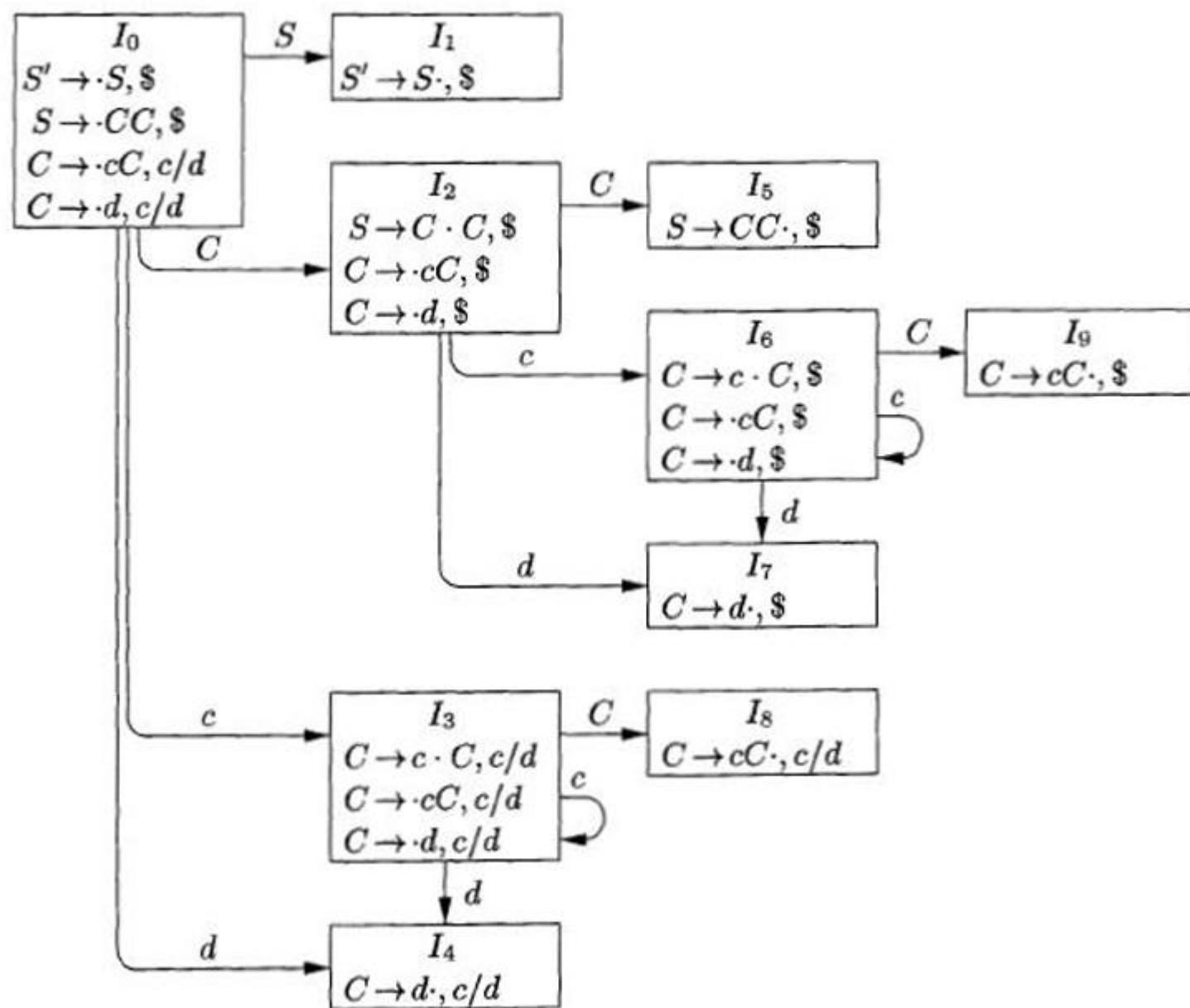
$C \rightarrow \bullet cC, \$$

$C \rightarrow \bullet d, \$$

$I_7 = \text{GOTO}(I_6, d)$

$C \rightarrow d\bullet, \$$

GOTO GRAPH FOR THE GRAMMAR



NUMBER THE PRODUCTIONS

1. $S \rightarrow CC$
2. $C \rightarrow cC$
3. $C \rightarrow d$

PARSING TABLE

$$I_1 = S' \rightarrow S \bullet, \$$$

$$I_4 = C \rightarrow d \bullet, c \mid d$$

$$I_5 = S \rightarrow CC \bullet, \$$$

$$I_7 = C \rightarrow d \bullet, \$$$

$$I_8 = C \rightarrow c C \bullet, c \mid d$$

$$I_9 = C \rightarrow c C \bullet, \$$$

STATE	ACTION			GOTO	
	<i>c</i>	<i>d</i>	<i>\$</i>	<i>S</i>	<i>C</i>
0	s3	s4		1	2
1			acc		
2	s6	s7			5
3	s3	s4			8
4	r3	r3			
5			r1		
6	s6	s7			9
7			r3		
8	r2	r2			
9			r2		

CONSTRUCTING LALR PARSING TABLE

- ✦ LALR stands for **Lookahead LR**.
- ✦ LALR parsers are often used in practice because LALR parsing tables are smaller than LR(1) parsing tables.
- ✦ The number of states in SLR and LALR parsing tables for a grammar G are equal.
- ✦ But LALR parsers recognize more grammars than SLR parsers.
- ✦ *yacc* creates a LALR parser for the given grammar.
- ✦ A state of LALR parser will be again a set of LR(1) items.

Creating LALR Parsing Tables

✚ Canonical LR(1) Parser $\rightarrow$ LALR Parser

The Core of A Set of LR(1) Items

✚ The core of a set of LR(1) items is the set of its first component

Ex: $S \rightarrow L.=R, \$ \rightarrow S \rightarrow L.=R$ Core

$R \rightarrow L., \$ \quad R \rightarrow L.$

- We will find the states (sets of LR(1) items) in a canonical LR(1) parser with same cores. Then we will merge them as a single state.

$I_1: L \rightarrow id., =$

A new state: $I_{12}: L \rightarrow id., =$

$\rightarrow$

$L \rightarrow id., \$$

$I_2: L \rightarrow id., \$$

have same core, merge them

- We will do this for all states of a canonical LR(1) parser to get the states of the LALR parser.
- In fact, the number of the states of the LALR parser for a grammar will be equal to the number of states of the SLR parser for that grammar.

We need to do following update in the parsing table.

- Consider a pair of similar looking states (same kernel and different lookaheads) in the set of LR(1) items
 $I_4: C \rightarrow d. , c/d$ $I_7: C \rightarrow d. , \$$
- Replace I_4 and I_7 by a new state I_{47} consisting of $(C \rightarrow d. , c/d/\$)$
- Similarly I_3 & I_6 and I_8 & I_9 form pairs
- Merge LR(1) items having the same core

INPUT: An augmented grammar G' .

OUTPUT: The LALR parsing – table functions ACTION and GOTO for G'

METHOD:

1. Construct $C = \{ I_0, I_1, \dots, I_n \}$, the collection of sets of LR(1) items.
2. For each core present among the set of LR(1) items, find all sets having that core, and replace these sets by their union.
3. Let $C' = \{ J_0, J_1, \dots, J_m \}$, the collection of sets of LR(1) items. The parsing action for state i are constructed from J_i in the same manner as in Algorithm. If there is a parsing action conflict, the algorithm fails to produce a parser, and the grammar is said not to be LALR(1)
4. The GOTO table is constructed as follows. if J is the union of one or more sets of LR(1) items, that is, $J = I_1 \cup I_2 \cup \dots \cup I_k$, then the $I_1, I_2, \dots, I_k$ all have the same core. Let K be the union of all sets of items having the same core as $\text{GOTO}(I_1, X)$. Then $\text{GOTO}(J, X) = K$.

STATE	ACTION			GOTO	
	<i>c</i>	<i>d</i>	\$	<i>S</i>	<i>C</i>
0	s36	s47		1	2
1			acc		
2	s36	s47			5
36	s36	s47			89
47	r3	r3	r3		
5			r1		
89	r2	r2	r2		

Let us see the comparison between SLR, CLR, and LALR Parser.

SLR Parser	LALR Parser	CLR Parser
It is very easy and cheap to implement.	It is also easy and cheap to implement.	It is expensive and difficult to implement.
SLR Parser is the smallest in size.	LALR and SLR have the same size. As they have less number of states.	CLR Parser is the largest. As the number of states is very large.
Error detection is not immediate in SLR.	Error detection is not immediate in LALR.	Error detection can be done immediately in CLR Parser.
SLR fails to produce a parsing table for a certain class of grammars.	It is intermediate in power between SLR and CLR i.e., $SLR \leq LALR \leq CLR$.	It is very powerful and works on a large class of grammar.
It requires less time and space complexity.	It requires more time and space complexity.	It also requires more time and space complexity.